Working With AI

A practical handbook — how these models actually work, how to prompt them, which tools to use, and how to get consistently better answers.

Companion notes to the AI & Data Science session · Prepared by Ranjith · September 2026
Model names and tool prices change every few weeks. The *mechanics* in Part 1 and the *method* in Part 3 do not. Learn those; look up the rest.

Contents

1 · How the machine actually works

Everything in this handbook follows from one sentence. If you only remember one line, remember this:

> THE ONE FACT
>
> **A language model predicts the next piece of text. It does not look anything up.**
>
> Every strength (fluency, speed, flexibility) and every failure (made-up facts, bad maths, confident nonsense) comes from this single behaviour.

1.1 The nesting — what sits inside what

Layer	What it means	Example
AI	Any machine doing something that normally needs human intelligence.	A chess engine. A spam filter.
Machine Learning	The machine learns rules from examples instead of being told the rules.	Predicting house prices from past sales.
Deep Learning	ML using neural networks with many layers. Handles messy data — images, audio, text.	Face recognition.
Generative AI	Deep learning that *produces* new content rather than labelling existing content.	ChatGPT, Midjourney.
LLM	Generative AI specialised in text. A Large Language Model.	Claude, GPT, Gemini.

The distinction that matters: a *discriminative* model answers "which bucket does this belong to?" (spam / not spam). A *generative* model answers "what comes next?" That is a much harder job, and it is why generative models sometimes invent things — inventing is literally the mechanism.

1.2 Read the name backwards

Word	What it actually tells you
Model	A giant mathematical function. Text in → probabilities out. Nothing more mystical than that.
Language	Its only medium is text (and now images/audio converted into the same format). It has no senses, no body, no live view of the world.
Large	Hundreds of billions of adjustable numbers ("parameters"), trained on a large slice of public text. Large enough that useful behaviour *emerges* that nobody explicitly programmed.

1.3 Tokens — it does not see words

Before anything happens, your text is chopped into **tokens**: common chunks of characters. Roughly **1 token ≈ 4 English characters ≈ ¾ of a word**.

- `unbelievable` might become `un | bel | iev | able` — four tokens, and the model never sees the whole word.
- Indian languages usually cost **2–4× more tokens** than the same sentence in English, because the tokenizer was built mostly on English text. Same meaning, higher cost, less room in the context window.
- This is why models historically fumbled letter-counting and spelling puzzles — they were never looking at letters.

See it yourself: `platform.openai.com/tokenizer` — paste an English sentence and its Tamil/Hindi translation and compare the counts.

1.4 Training — what it kept and what it lost

Stage	What happens
Pre-training	The model reads an enormous amount of text and repeatedly plays one game: hide the next token, guess it, adjust the numbers when wrong. Billions of times. It ends up a brilliant, amoral autocomplete.
Fine-tuning	Further training on curated examples of good answers, so it behaves like an assistant instead of continuing your sentence.
RLHF Reinforcement Learning from Human Feedback	Humans rank answers; the model is nudged toward the ones people preferred. This is where politeness, helpfulness and refusals come from — *and* where sycophancy comes from. It was rewarded for answers people *liked*, which is not identical to answers that are *correct*.

CONSEQUENCE

The training text is compressed into the weights the way a summary compresses a book. **Patterns survive. Exact facts often do not.** The model retains the *shape* of a citation, a phone number, an API call — and will happily produce a shape that is plausible and wrong.

1.5 The context window is its entire world

- The **context window** is everything it can see in one go: system instructions + your message + uploaded files + the whole conversation so far.
- Sizes today range from ~128,000 tokens to 1,000,000+ tokens (roughly 100 to 1,500 pages).
- **It has no memory between chats.** A new chat starts from zero. "Remember what I told you yesterday" only works if the product is re-feeding it to the model behind the scenes.

- When a conversation overflows the window, the earliest parts get dropped or summarised — which is why very long chats start "forgetting" your instructions.
- Attention is not uniform. Material at the very start and very end of a long context is used more reliably than material buried in the middle. **Put your instruction after the pasted document, not before it.**

Test it in 20 seconds: open a fresh chat and ask "what did I ask you last time?" Watch it have no idea.

1.6 Temperature and why answers change

At each step the model has a probability distribution over possible next tokens. **Temperature** decides how adventurously it picks.

Setting	Behaviour	Use for
Low (0 – 0.3)	Almost always takes the highest-probability token. Repeatable, dry.	Code, data extraction, classification, maths, anything factual.
Medium (0.7)	Default in most chat products. Balanced.	General writing, explanation, conversation.
High (1.0+)	Samples unusual tokens. Surprising, sometimes incoherent.	Brainstorming, names, creative variants.

You can set this in an API or playground, not usually in the consumer chat app. It explains why the same prompt gives different answers twice — and why "be creative" and "be accurate" pull in opposite directions.

1.7 Why it hallucinates — the payoff

A **hallucination** is a fluent, confident, false statement. It is not a bug that will be patched away. It is the mechanism working as designed:

1. The model must output *something* — there is no "I don't know" token by default.
2. It was trained to produce text that *looks like* a correct answer.
3. A fabricated citation looks exactly like a real one, statistically.
4. RLHF taught it that confident answers score better than hedged ones.

Highest risk	Lowest risk
Specific numbers, dates, statistics	Explaining a well-known concept
Citations, paper titles, page numbers	Restructuring text you supplied
Names of people at a company	Summarising a document you pasted
Legal sections, medical dosages	Common code patterns
Niche API parameters and flags	Brainstorming and naming
Anything after its training cutoff	Translation, tone change, formatting

THE RULE THAT FOLLOWS

Never ask it to recall. Always give it the source and ask it to reason.

"Summarise this paper" (paper attached) is safe. "What does that paper say?" is not.

1.8 Grounding — the fix

Putting real information *into the context window* so the model reasons over facts instead of memories. Three ways, cheapest first:

Method	How	When

Paste	Drop the text straight into the prompt.	A few pages. Always try this first.
Upload / Project files	Attach PDFs; the product handles chunking and retrieval.	A syllabus, a report, a set of notes.
RAG Retrieval-Augmented Generation	Documents are embedded into a vector database; the most relevant chunks are fetched and injected at question time.	Thousands of documents, or a product you're building.
Web search	The tool fetches live pages and feeds them in.	Anything recent. Check the links it cites.

2 · The model landscape

2.1 The main families

Version numbers move fast — treat the *families* as stable and check the provider's page for the current release.

Family	Maker	Open?	Character, in practice
Claude	Anthropic	Closed	Strong at long documents, careful reasoning, writing and coding. Tiers: Opus (most capable) · Sonnet (balanced, the workhorse) · Haiku (fast, cheap).
GPT	OpenAI	Closed	The broadest ecosystem — images, voice, custom GPTs, huge plugin surface. Most people's default.
Gemini	Google	Closed	Deeply wired into Search, Docs, Gmail, Android. Very large context. Generous free API tier — the reason it's used for student projects.
Llama	Meta	Open weights	Download and run it yourself. The base for most self-hosted work.
Mistral	Mistral AI	Open + closed	Small, fast, efficient models. Good on a laptop.
DeepSeek / Qwen	DeepSeek · Alibaba	Open weights	Strong reasoning and coding at very low cost. Qwen has good multilingual coverage.
Grok	xAI	Mixed	Tied to X/Twitter data and real-time posts.

2.2 The three axes that actually decide your choice

Axis	What it means for you
Fast vs reasoning	"Reasoning" models think through steps internally before answering — slower and dearer, much better at maths, logic, multi-step code and planning. Fast models are better for drafting, summarising, chat, rewriting. Use the cheap fast one by default; escalate when it fails.
Open vs closed	*Closed* = you call their API, they run it, you pay per token, your data leaves your machine. *Open weights* = you download and run it on your own hardware; free, private, and entirely your problem to operate.
Text vs multimodal	Multimodal models accept images, PDFs, audio, sometimes video. Screenshot a whiteboard, a graph, an error message, a handwritten page — all valid input now.

2.3 Which one for which job

Task	Reach for	Why
Long PDF, notes, research paper	Claude, Gemini	Large context, reliable over long documents.
Writing and editing code	Claude, GPT reasoning tier	Best code quality; strong at debugging from an error trace.
Maths, logic, algorithms	Any reasoning model	Internal step-by-step beats one-shot guessing.
Anything from this week	Any tool with web search on	Training cutoffs are months old. No search = no recent facts.

Free student project / API	Gemini (Google AI Studio)	Free key, no credit card, adequate limits.
Private or offline data	Llama / Mistral via Ollama	Runs locally. Nothing leaves your laptop.
Images from text	Midjourney, DALL·E, Stable Diffusion, Imagen	Different aesthetics — try the same prompt on each.

HABIT WORTH INSTALLING

For any question that matters, run it past **two different families** (say Claude and Gemini). Where they agree, confidence is high. Where they disagree, you have found exactly the thing you need to go and verify yourself.

3 · Prompting — the core skill

3.1 What prompting actually is

A prompt is not a search query and not a magic spell. It is the model's *entire universe* for that one answer — it knows nothing about you, your subject, your college or your intent beyond what is written in the window.

> WORKING DEFINITION
>
> **Prompting is the job of making a wrong answer statistically unlikely.**
>
> You are not persuading it. You are narrowing the space of plausible next tokens until only good answers remain.

This is why the most common complaint — "*I didn't get the answer I expected*" — is almost never the tool's fault. The expectation lived in your head and never made it into the window.

3.2 The anatomy of a good prompt — six parts

Part	What it does	Example line
1. Role	Sets vocabulary and depth. Mild effect, but free.	"You are a database examiner marking second-year answers."
2. Task	One clear verb. The single most important part.	"Rewrite this answer so it would score full marks."
3. Context	Who it's for, what's already been tried, what the constraints are.	"It's for a 10-mark question. We've covered indexing but not query planning."
4. Input	The actual material — pasted, not remembered. Delimit it clearly.	`<answer> … </answer>`
5. Format	Exactly how the output should be shaped.	"Return a table: Gap | Why it loses marks | Corrected line."
6. Constraints	The boundaries — length, what to avoid, what to do when unsure.	"Under 200 words. No new facts. If the answer is already correct, say so and stop."

You will not need all six every time. **Task + Context + Input** covers most of the value. Add Format when the shape of the output matters; add Constraints when it keeps going off the rails.

3.3 Before and after

Weak prompt

```
explain normalization
```

→ A textbook definition you could have got from any website. No use to you.

Strong prompt

```
You are tutoring a 2nd-year AI & Data Science student.

Explain gradient descent variants — batch, mini-batch and stochastic.

Context: I understand what a derivative is and I've trained one linear
regression model by hand. I do not know what a "batch" means here.

Format:
1. One plain-English paragraph per variant, no equations.
2. Then a table: Variant | Data used per step | Speed | Stability | When to use.
3. End with the single sentence I should memorise for an exam.

Constraints: under 400 words. If a term is unavoidable, define it in
brackets the first time. Do not use the word "simply".
```

→ Tuned to your level, shaped to how you revise, and checkable at a glance.

3.4 The seven rules

#	Rule	Why it works
1	**Give it the material.**	Grounded reasoning beats compressed memory. Paste the notes, the error, the code, the brief.
2	**Say who it's for.**	"Explain to a 2nd-year student" and "explain to a professor" produce genuinely different text.
3	**Specify the output shape.**	Table, bullets, JSON, 5 lines. Unspecified shape = generic essay.
4	**One task per prompt.**	Four questions in one message get four shallow answers. Chain instead.
5	**Show an example.**	One sample of the output you want beats three paragraphs describing it.
6	**Say what *not* to do.**	"No preamble." "Don't apologise." "Don't invent sources." Negative constraints are cheap and effective.
7	**Iterate, don't restart.**	The context you've built is an asset. "Good — now make it shorter and add a counter-example" beats retyping from scratch.

3.5 Anti-patterns — stop doing these

Habit	What goes wrong	Do instead
Typing it like a Google search	Three keywords give a generic article.	Write a sentence with a verb and a goal.
"Make it better"	Better on which axis? It guesses.	"Shorter, more concrete, drop the intro."
Asking it to recall facts	Confident fabrication.	Attach the source. Or turn search on.
Stuffing five tasks into one message	All five done badly.	One task; then the next.
Starting a new chat after a bad answer	You throw away all the context you built.	Correct it in place.
"Please" / "you are the world's best expert"	Politeness costs nothing but adds nothing. Superlatives don't unlock hidden ability.	Spend those words on context instead.
Accepting the first answer	The first draft is a draft.	Push back once. It usually improves.

Pasting code with no error message	It guesses at what broke.	Paste the full trace, the input, and what you expected.

4 · Prompting techniques, named

These have formal names in papers. Here they are with the one line that tells you when to reach for each.

Technique	What you do	Use when
Zero-shot	Just ask. No examples.	Common, well-defined tasks. Your default.
Few-shot	Show 2–5 input→output examples, then give the real input.	The output format is specific or unusual. The single highest-leverage technique for consistency.
Chain of thought	"Work through this step by step before giving the final answer."	Maths, logic, multi-step problems. Forcing intermediate steps into the text genuinely improves accuracy. (Reasoning models do this internally already.)
Role / persona	"You are an examiner / a code reviewer / a sceptical investor."	You want a particular vocabulary, depth or stance.
Decomposition	Split a big task into numbered sub-tasks; run them one at a time.	Anything that fails as a single prompt.
Prompt chaining	Feed the output of prompt 1 into prompt 2. Outline → draft → tighten.	Long or structured deliverables. Better than one giant prompt, every time.
Self-critique	"Now list three weaknesses in what you just wrote, then rewrite fixing them."	Free quality upgrade on any draft. Takes ten seconds.
Contrastive	"Give me two versions: one formal, one blunt."	You don't yet know what you want. Pick, then refine.
Structured output	Demand JSON / CSV / a fixed table with named columns.	Another program will consume the output.
Delimiters	Wrap pasted material in `<notes>…</notes>` or triple quotes.	Always, when pasting. Stops it confusing your data with your instructions.
Instruction-last	Put the long document first, your instruction at the very end.	Long contexts. The end of the window gets the most attention.
Escape hatch	"If the notes don't cover this, say 'not in the notes' — do not guess."	Always, on factual work. This is the cheapest anti-hallucination line there is.
Interview me	"Before answering, ask me the 5 questions you most need answered."	Vague or personal tasks — project ideas, resumes, plans. Flips the effort onto the model.
Rubric	Give it the marking scheme, then ask it to grade its own output against it.	Assignments, reports, anything with known criteria.
ReAct	Reason → Act (call a tool) → Observe → reason again.	The pattern underneath every AI agent. See Part 9.

4.1 Three you can copy today

The self-critique loop

```
[your normal prompt]

Then, in a separate section titled "Critique":
list the 3 weakest things about your own answer above.
Then rewrite the answer fixing all 3.
```

The interview flip

```
I want to build a final-year project using AI.

Do not suggest anything yet. First ask me the 6 questions you most
need answered to give a good recommendation. Ask them one at a time.
```

The grounded answer

```
<notes>
[paste your unit notes here]
</notes>

Using ONLY the notes above, answer: [question]

Quote the exact line you used, in brackets, after each claim.
If the notes do not cover it, reply "Not in the notes" and stop.
```

5 · Debugging a bad answer

When the output is wrong, it's a diagnosis, not a dead end. Work down this table.

Symptom	Actual cause	Fix
Generic, textbook-ish, useless	No context — it doesn't know your level or situation.	Add who it's for and what you already know.
Too long, padded, waffly	No length or format constraint.	"Max 150 words. Bullets only. No intro."
Made-up facts or citations	You asked it to recall.	Paste the source, or turn on web search, and add the escape hatch line.
Wrong format every time	Described instead of demonstrated.	Show one example of the exact output.
Answers a different question	Ambiguous verb, or several tasks at once.	One task, one clear verb.
Agrees with everything you say	Sycophancy from RLHF.	"Argue the opposite case." "What's wrong with my reasoning?"
Forgot your instructions mid-chat	Context window filling up; early text de-prioritised.	Restate the constraint in your latest message, or start fresh with a clean brief.
Maths or logic wrong	It's predicting text, not calculating.	Ask for step-by-step, use a reasoning model, or make it write code to compute it.
Code doesn't run	It guessed at a library version or API.	Paste the full error trace back. Say the language and version.

Bland, safe, obvious ideas	You asked for "ideas" with no constraints.	Add hard constraints — they force originality. "Under ₹0 budget, in 2 weeks, no app."

6 · How to treat AI better

Not manners — *method*. These are the habits that separate people who find AI useful from people who find it disappointing.

Habit	What it looks like
Brief it like a colleague	You wouldn't say "write my report" to a new intern and walk away. Give background, audience, constraints and an example of good. Same effort, same payoff.
Treat it as a draft engine	It gets you from a blank page to 70% in one minute. The last 30% — judgement, accuracy, your voice — is yours and always will be.
Argue with it	"That's not right because…" is the fastest route to a good answer. It will not sulk. Pushing back once is the single highest-return habit in this handbook.
Ask it to think, not to know	Reasoning over material you supplied is its strength. Recall is its weakness. Structure every task to use the first.
Make it ask you things	"What do you need to know before answering?" Most bad answers come from missing context the model never asked for.
Verify anything that carries a consequence	Numbers, citations, legal or medical claims, code that touches real data. Confidence in the text is not evidence.
Keep your prompts	A note file of prompts that worked is a real asset. Reuse and refine them instead of rewriting from memory.
Use it to learn, not to skip	"Explain this like I'm 10, then quiz me on it" builds understanding. Copy-pasting an answer you can't defend builds nothing — and it shows in the viva.
Keep the human in the loop	You are accountable for the output. Not the model, not the vendor. Never ship what you can't explain.
Name what you don't want	Bans are as useful as instructions. "No em-dashes, no 'delve', no summary paragraph at the end."

THE MENTAL MODEL

Treat it as a **brilliant, extremely well-read intern with no memory, no judgement and total confidence**. That single frame predicts almost every behaviour you'll encounter — good and bad.

7 · The tool map

Free tiers and prices change constantly — verify before you rely on any of them.

7.1 Chat assistants — start here

Tool	What it's best at	Notes
ChatGPT	General everything. Widest feature surface.	Free tier is usable. Custom GPTs, voice, image generation.
Claude	Long documents, careful writing, coding.	Projects hold persistent files and instructions.

Gemini	Google-ecosystem work, huge context.	Free API key for projects via Google AI Studio.
Perplexity	Research with live sources and citations.	Use when you need links you can actually check.
NotebookLM	Q&A strictly over documents you upload.	Answers stay grounded in your sources. Excellent for exam revision.

7.2 Coding

Tool	What it is	Notes
GitHub Copilot	Autocomplete inside your editor.	**Free for students** via the GitHub Student Developer Pack.
Cursor	A code editor built around AI — edits across multiple files.	The jump from "suggests a line" to "makes the change".
Claude Code	AI agent that works in your terminal on a whole repo.	Runs commands, edits files, fixes its own errors.
Windsurf, Cline	Similar agentic editors.	Worth trying; the space moves monthly.
v0, Bolt, Lovable	Prompt → working web app.	Fast prototypes and demos.
Google Colab	Free notebooks with a free GPU.	Where most student ML projects should live.

7.3 Building agents and automations

Tool	What it is	Notes
n8n	Visual workflow builder with AI agent nodes.	No coding floor. Self-hostable and free. Best first tool for building an agent.
Zapier / Make	Connect apps, trigger on events.	Simpler, less flexible, quicker to a result.
LangChain / LlamaIndex	Python frameworks for agents and RAG.	For when you've outgrown the visual tools.
Ollama	Run open models locally on your laptop.	Free, private, offline. Needs decent RAM.
Hugging Face	The repository of open models and datasets.	Where you'll go for anything you want to fine-tune.

7.4 Media and study

Category	Tools	Use for
Images	Midjourney, DALL·E, Stable Diffusion, Imagen, Ideogram	Slides, posters, mockups. Ideogram handles text-in-image best.
Video	Runway, Pika, Sora, Veo	Short clips, b-roll, concept films.
Voice / audio	ElevenLabs, Whisper	Narration; transcribing recorded lectures (Whisper is free and open).
Design	Figma AI, Canva Magic Studio	Layouts, decks, quick assets.
Notes / study	NotebookLM, Notion AI, Obsidian + Copilot	Turning your own notes into questions and summaries.

Papers	Elicit, Consensus, SciSpace	Literature search that cites real papers.

8 · Beyond the chat box

Four ways to reach the same model. Most people only ever use the first.

Route	What it gives you	Trade-off
Chat app	Fastest start. Files, search, images built in.	Manual. Nothing is reusable.
Projects / Custom GPTs	Persistent instructions + files across many chats. You stop re-explaining yourself.	Still you, typing. Set one up for each subject.
API	Call the model from code. Control temperature, system prompt, structured output. Put AI inside your own app.	Needs a key; needs code; usage costs money (Gemini's free tier is the student exception).
Agent platform	The model gets tools and a loop, and acts without you typing each step.	Setup effort. Real failure modes. See Part 9.

8.1 Two things worth setting up this week

- **Custom instructions.** Written once, applied to every chat. Mine should say: your course, your year, how you like explanations, what to never do. Ten minutes; improves every answer you get after it.
- **A Project per subject.** Upload the syllabus, the notes, past papers. Now every question you ask is automatically grounded in *your* material, not the internet's average.

8.2 MCP — connecting AI to your real tools

Model Context Protocol is an open standard that lets an AI assistant talk to external systems — your files, a database, GitHub, Google Drive — through a common interface instead of a bespoke integration per app. In practice: the assistant can read and act on your real data. Worth knowing the name; the space is standardising around it.

9 · Agents — what they really are

THE CORRECTION MOST PEOPLE NEED

Adding tools to a chatbot does not make it an agent.

The test: *does it do multi-step work toward a goal while you are not typing?* If you're in the loop every turn, it's a chatbot with plugins.

9.1 The honest ladder

#	Level	What it does	Agent?
1	Chatbot	You ask, it answers from training.	No
2	Chatbot + RAG	Answers from your documents.	No
3	Chatbot + tools	Can search the web, run a calculation — but only when you ask, one step at a time.	No

4	**Agent**	Takes a *goal*. Plans, acts, checks the result, re-plans, and delivers — without you between steps.	**Yes**
5	Multi-agent	Several agents with different roles coordinating on one goal.	Yes

9.2 The loop

```
GOAL  →  THINK  →  ACT  →  OBSERVE  →  THINK AGAIN  →  …  →  DELIVER
                                            ↑
                              this step is the whole difference
```

"Think again" is where it decides the last result wasn't good enough and changes approach. A system that can't do that is a script with a language model bolted on.

9.3 The four components

Component	Role
Model	The brain. Decides what to do next.
Tools	The hands. Web search, a database, a calculator, an email send, a spreadsheet write.
Loop	The thing that lets it keep going until the goal is met rather than stopping after one reply.
Memory	What carries across steps and across runs — a file, a sheet, a database.

9.4 Worked example — "Unit Prep Agent"

You type once: "*Prepare me for Unit 3.*" Then you walk away.

1. Reads your uploaded unit notes.
2. Identifies every topic in the unit.
3. **Judges** which topics the notes cover thinly — a real decision, not a fixed script.
4. Looks those up on Wikipedia.
5. Writes one row per weak topic to a Google Sheet, each with a practice question.
6. Reports back what it found.

Add a schedule trigger at 8pm and it runs while you're not there. **That** is the agent moment.

THE TWO LINES THAT CREATE AGENCY

`Work through the entire goal before replying.`
`Do NOT ask permission between steps.`

Delete them and it collapses back into a chatbot.

9.5 Where agents break

Failure	What to do about it
Loops forever	Cap the iterations. Always.
Picks the wrong tool	Sharpen the tool descriptions — that text is what it uses to choose.
One bad step poisons the rest	Add a verification step; make it state its assumption before acting.

Takes a destructive action	Require human approval for anything irreversible — sending, deleting, paying.
Silent failure	Log every step. An agent you can't inspect is an agent you can't trust.

10 · Limits, honesty and privacy

10.1 What it genuinely cannot do

Limit	What it means in practice
No memory between chats	Every new conversation starts blank unless the product re-feeds context.
Training cutoff	It knows nothing after a fixed date. Without search, recent questions get confidently outdated answers.
No real arithmetic	It predicts what a calculation looks like. For anything that matters, make it write code.
No access to your world	No view of your files, your college portal or your marks unless you give it one.
No self-knowledge	It cannot reliably tell you why it produced an answer. Its explanation is also generated text.
Inherited bias	It learned from human text, with all the skew that carries.

10.2 Privacy and cost — the practical rules

- Assume anything you paste leaves your machine. Free consumer tiers may use conversations to improve models; business and API tiers typically don't. **Check the setting — don't assume.**
- Never paste: passwords, API keys, Aadhaar or ID numbers, bank details, other people's personal data, or anything under an NDA.
- Anonymise before you paste. Real analysis rarely needs real names.
- For genuinely sensitive data, run a local model (Ollama). Nothing leaves the laptop.
- **Never share one API key across a group.** Free-tier limits are per key — one shared key throttles everybody. Each person gets their own; it takes about a minute.

10.3 Academic honesty

Fine	Not fine
Explaining a concept you're stuck on Generating practice questions Reviewing and critiquing your own draft Debugging your code Getting unstuck on a blank page Summarising papers you've read	Submitting generated work as your own Anything your institution prohibits Any output you can't explain in a viva Fabricated citations in a report Using it where you were told not to

The practical test: **can you defend every line of it out loud, without notes?** If not, you don't understand it yet, and that gap will surface.

11 · Your first 30 days

Week	Do this	You should end up able to

Week 1 Foundations	Set custom instructions. Create a Project per subject and upload your notes. Run the tokenizer test. Run the empty-chat memory test.	Explain tokens, context and hallucination to a classmate.
Week 2 Prompting	Rewrite five of your usual prompts using the six-part anatomy. Use the self-critique loop daily. Start a prompt library file.	Get a usable answer first try, most of the time.
Week 3 Grounding & tools	Upload a real paper and question it with the escape-hatch prompt. Try Perplexity and NotebookLM. Get Copilot free with the student pack.	Never accept an unsourced fact again.
Week 4 Build	Get a Gemini API key. Build one n8n workflow end to end. Then add a tool and a schedule trigger.	Point at something running that you built.

11.1 Exercises worth doing

1. Take your worst recent prompt. Rewrite it with all six parts. Compare the two outputs side by side.
2. Ask the same question to three models. Note where they disagree — then go and find out who was right.
3. Ask about something you know deeply. Count the errors. This recalibrates your trust permanently.
4. Upload a paper you've actually read. Ask five questions. Check every answer against the text.
5. Make it grade your own assignment against the real rubric before you submit.
6. Build an agent that does one boring thing for you every day at a fixed time.

12 · One-page cheat sheet

The prompt skeleton

```
ROLE        You are a [role] helping a [audience].
TASK        [One clear verb + object.]
CONTEXT     Here's what I already know / have tried / am constrained by.
INPUT       <material> … </material>
FORMAT      Return: [table with these columns / N bullets / JSON].
CONSTRAINTS Under [N] words. Don't [X]. If you can't do it from the
            material above, say so instead of guessing.
```

Ten lines that fix most answers

Say this	To get this
"Ask me what you need to know first."	Context you forgot to give.
"Think step by step before answering."	Better reasoning.
"Only use the text I gave you."	No invented facts.
"Say 'not in the source' if it isn't there."	An honest gap instead of a fabrication.
"Give me two versions, different approaches."	A real choice.
"Now critique your own answer, then rewrite it."	A free second draft.
"Argue against this."	The flaw you couldn't see.
"Explain it like I'm 10, then quiz me."	Actual understanding.
"Return it as a table with these columns: …"	Something usable.
"Shorter. No preamble. Start with the answer."	Less waffle.

The five rules, compressed

1. It predicts text — it does not look things up.
2. Give it the material; never ask it to recall.
3. Context and format beat clever wording, every time.
4. Iterate in place. Never restart after one bad answer.
5. Verify anything with a consequence. You are accountable, not the model.

13 · Glossary

Term	Meaning
Agent	A system that takes a goal and works through multiple steps, using tools, without you prompting each step.
API	A way to call the model from your own code instead of a chat window.
Context window	Everything the model can see at once. Its entire world for that answer.

Embedding	Text converted into a list of numbers so that similar meanings sit close together. The basis of search and RAG.
Few-shot	Giving examples of the output you want inside the prompt.
Fine-tuning	Further training on your own examples to change default behaviour. Rarely the right first answer — try prompting and grounding first.
Grounding	Supplying real source material so the model reasons over facts rather than memory.
Hallucination	Confident, fluent, false output. A feature of the mechanism, not a bug.
Inference	Running the trained model to get an answer (as opposed to training it).
LLM	Large Language Model. A model trained to predict the next token of text.
MCP	Model Context Protocol — an open standard for connecting AI assistants to external tools and data.
Multimodal	Accepts more than text — images, audio, PDFs, video.
Parameters	The adjustable numbers inside the model. "70B" means 70 billion of them.
Prompt	Everything you send. The model's complete brief.
RAG	Retrieval-Augmented Generation — fetch relevant documents, then answer using them.
Reasoning model	A model that thinks through steps internally before replying. Slower, dearer, much better at hard problems.
RLHF	Reinforcement Learning from Human Feedback — training on human preferences. Source of helpfulness, and of sycophancy.
System prompt	Hidden standing instructions that apply to every message in a conversation.
Temperature	Randomness dial. Low = predictable, high = creative.
Token	A chunk of text, roughly ¾ of a word. The unit the model reads, writes and bills in.
Training cutoff	The date after which the model knows nothing.
Weights	The trained parameters. "Open weights" means you can download and run them yourself.
Zero-shot	Asking with no examples.

Everything here follows from one line: *a language model predicts the next piece of text; it does not look anything up.* Learn that, and the rest is practice.